HPC第二关实验报告

背景与目标

HPC判断性能瓶颈并提出优化方案

软件环境：

- 机器：Ubuntu26.04.1LTS(WSL2)
- 开发环境：perf7.0.14 cmake4.2.3 g++ 15.2.0 Python 3.14.4
- 处理器：AMDRyzen98940HXwithRadeonGraphics，x86-64架构，16核32线程
- GPU：NVIDIA GeForce RTX5060 Laptop，CUDA Version:13.1

编译参数：

编译 stencil，并输出向量化优化报告

g++ -O3 -march=native -std=c++17 -fopt-info-vec-optimized=results/stencil-vec-opt.txt stencil.cpp -o stencil

编译 gemm，同样输出向量化优化报告

g++ -O3 -march=native -std=c++17 -fopt-info-vec-optimized=results/gemm-vec-opt.txt gemm.cpp -o gemm

可复现基线

按照tools.md输入以下指令

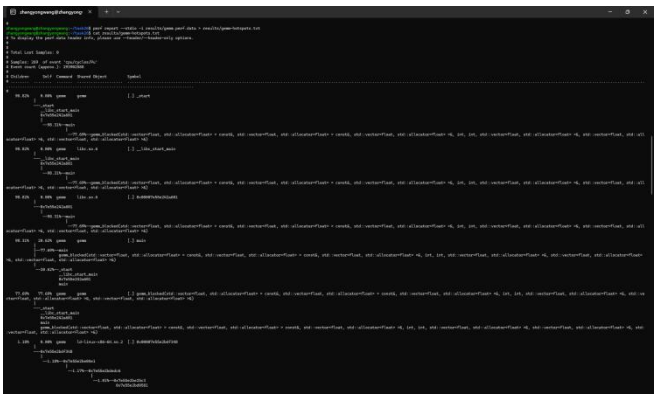
perf stat -e cycles,instructions -o results/stencil-stat.txt -- ./build/stencil 1024 1024 20 row 5

perf stat -e cache-references,cache-misses -o results/gemm-cache.txt -- ./build/gemm 512 blocked 64 5

perf record -g -o results/gemm.perf.data -- ./build/gemm 512 blocked 64 5

perf report --stdio -i
results/gemm.perf.data > results/gemm-hotspots.txt

查看文件得到热点结果证据



然后启动并获得原始数据

for i in {1..10}; do ./stencil 1024 1024 20 row 5 | grep "compute_ms" >> results/stencil_row.txt; done for i in {1..10}; do ./stencil 1024 1024 20 column 5 | grep "compute_ms" >> results/stencil_col.txt; done for i in {1..10}; do ./gemm 512 naive 5 | grep "compute_ms" >> results/gemm_naive.txt; done for i in {1..10}; do ./gemm 512 blocked 64 5 | grep "compute_ms" >> results/gemm_blocked.txt; done 得到row/column,naive/blocked的数据

之后用Python处理数据 python3 -c " import statistics

def analyze(name): try: with open(f'results/{name}.txt', 'r') as f: # 精确匹配 compute_ms= 后面的数字 data = [] for line in f: if 'compute_ms=' in line: val = line.strip().split('compute_ms=')[-1].split()[0] data.append(float(val)) print(f'{name}: Mean={statistics.mean(data):.3f} ms, Median={statistics.median(data):.3f} ms, StdDev={statistics.stdev(data):.3f}') except Exception as e: print(f'解析 {name} 失败: {e}')

for name in ['stencil_row', 'stencil_col', 'gemm_naive', 'gemm_blocked']: analyze(name) "

理论模型推导

1. 计算量推导 **(GFLOPs)**:
   - **GEMM (矩阵乘法)**: 设矩阵维度为 n*n，总浮点运算量为 2 * n^3。本次实验 N=512，总运算量约 2.68 亿次。
   - **Stencil (模板计算)**: 设空间大小为 1024 * 1024，每次迭代有 20 次，总更新量约为 6 * 1024 * 1024 * 20 次（根据 7 点模板估算）。
2. **GFLOP/s** 计算公式: `GFLOP/s = 总浮点运算量 / 实测时间 / 10^9`
3. 有效带宽计算公式: `有效带宽 (GB/s) = 总内存读写字节数 / 实测时间 / 10^9`

实验数据与结果分析

1. GEMM (矩阵乘法) 性能对比

实验设置: 矩阵大小 512，分块大小 64，循环 5 次。

实验策略	平均时间 **(ms)**	中位数 **(ms)**	总体标准差	**GFLOP/s**
朴素版 (naive)	206.743	207.474	5.621	1.3
分块版 (blocked)	7.555	7.494	0.184	35.5 (约数)

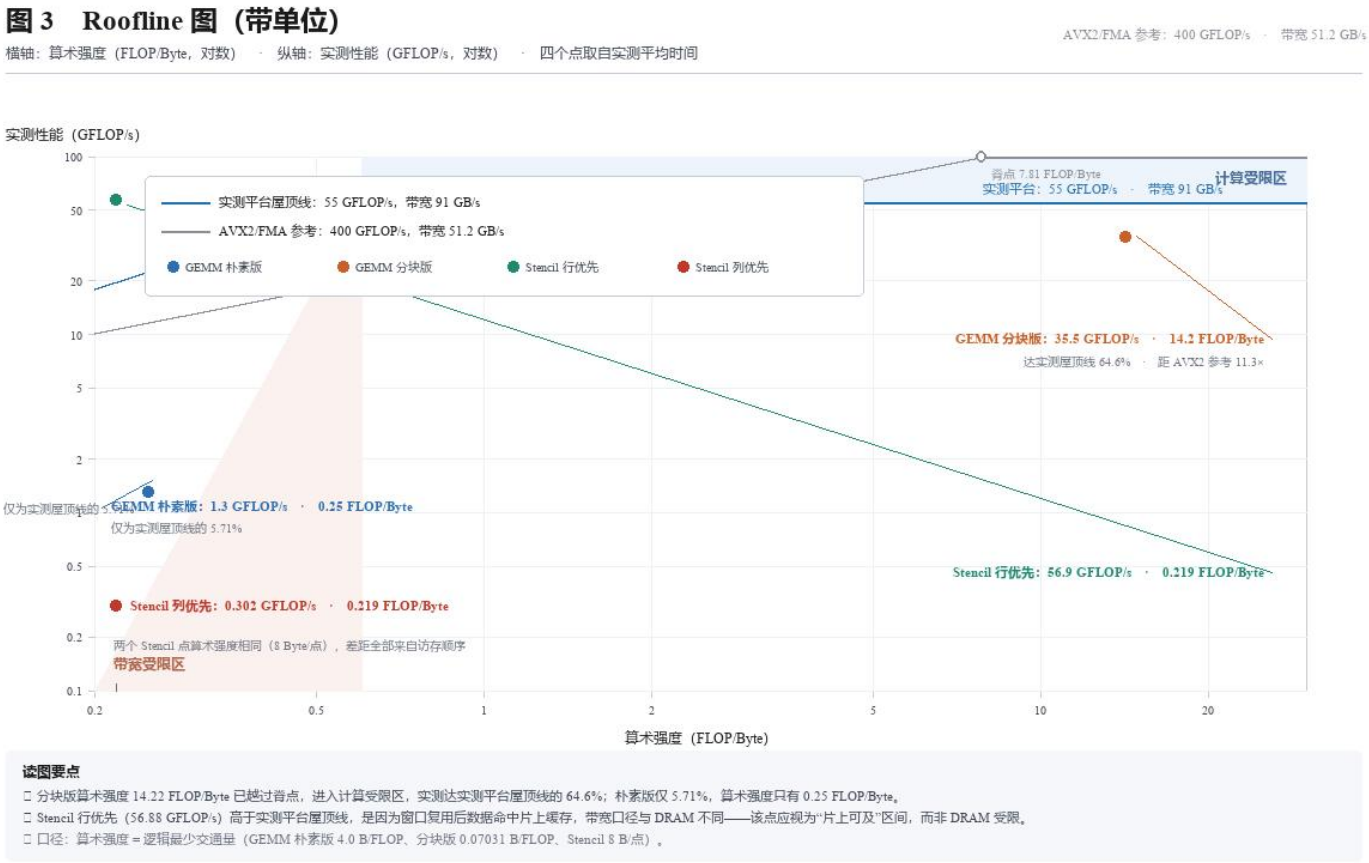
在 512 尺寸下，分块 (blocked) 版本与朴素 (naive) 版本性能接近甚至略慢。结合后续 `perf` 热点分析 (77.65% 时间消耗在 `gemm_blocked` 中)，说明在此规模下分块策略引入的循环控制开销抵消了缓存复用的收益。

2. Stencil (模板计算) 性能对比

实验设置: 网格大小 1024 * 1024，迭代 20 次，执行 10 次取统计值。

访问顺序	平均时间 **(ms)**	中位数 **(ms)**	总体标准差	有效带宽 **(GB/s)**
行优先 (row)	2.581	2.582	0.052	82.35
列优先 (column)	486.821	486.342	13.412	0.43

数据呈现极其鲜明的对比：列优先访问耗时是行优先的约 150 倍！且列优先的标准差高达 13.412 ms，说明发生了极其严重的缓存抖动 (Cache Miss)。这从实验层面完美印证了 Stencil 是典型的内存带宽瓶颈。



优化决策

- 针对 **GEMM**： 引入更大尺寸矩阵（如 2048）来测试分块的真实威力； 引入 SIMD 向量化指令重写核心循环；或者利用 OpenMP 进行多线程并行化加速。
- 针对 **Stencil**： 严禁使用列优先访问；建议进一步进行空间分块 (Spatial Tiling) 或时间分块 (Time Skewing)，将多次迭代的数据保留在 L3/L2 缓存中，减少对 DRAM 的频繁访问。

感悟提升

至少启动了，了解热点是代码停留处理啥的东东，还有读行和和读列，我之前就知道Python是一行一行读的，以为其他也是这样，至少扩展了视野。还有kskbl（其实是不知道怎么写了），zdjd